

M1 architecture: what the system is and what we decided

5 October 2026 | Specified design | Implementation validation pending

1. **N_intake** validates the request and snapshots permitted resources.
2. **N_intent** (one CC, **research.compile_intent**; a tool, rules only, no model call) derives what the request means. **G_intent** (model-backed verifier assessment after deterministic checks) checks it immediately.
3. **N_req** (requirement CC, **research.compile_brief**) derive outputs, constraints, acceptance and evidence from accepted intent. **G_req** checks each.
4. **N_plan** (planner service, not a CC) proposes a DAG of nodes, dependencies and typed data bindings from accepted requirements and a library snapshot.
5. **N_validate / N_bind** check the whole graph, choose exact CC versions, attach each node's Gate, **freeze**. Invalid plan = zero dispatch.
6. **N_dispatch** (SwarmFlow + supervisor) feeds declared inputs into the DAG and releases ready nodes.
7. **N_run** executes each node's CC locally. Output and capture are **committed first**, then **G_node** runs right after.
8. A failing Gate **halts** the run: no later or sibling capsule starts. Passing Gate + committed release unlocks successors.
9. **N_deliver** (ordinary code, not a capsule) assembles accepted terminal results, publishes, and returns them to the user.

SwarmFlow runs the fixed outer flow (all nine positions always exist). Planned nodes become fixed after step 5. M1 may run ready nodes one at a time. Steps 2-3 and every node call use the same runner and Gate path. See verification, runtime.

Capsule or not

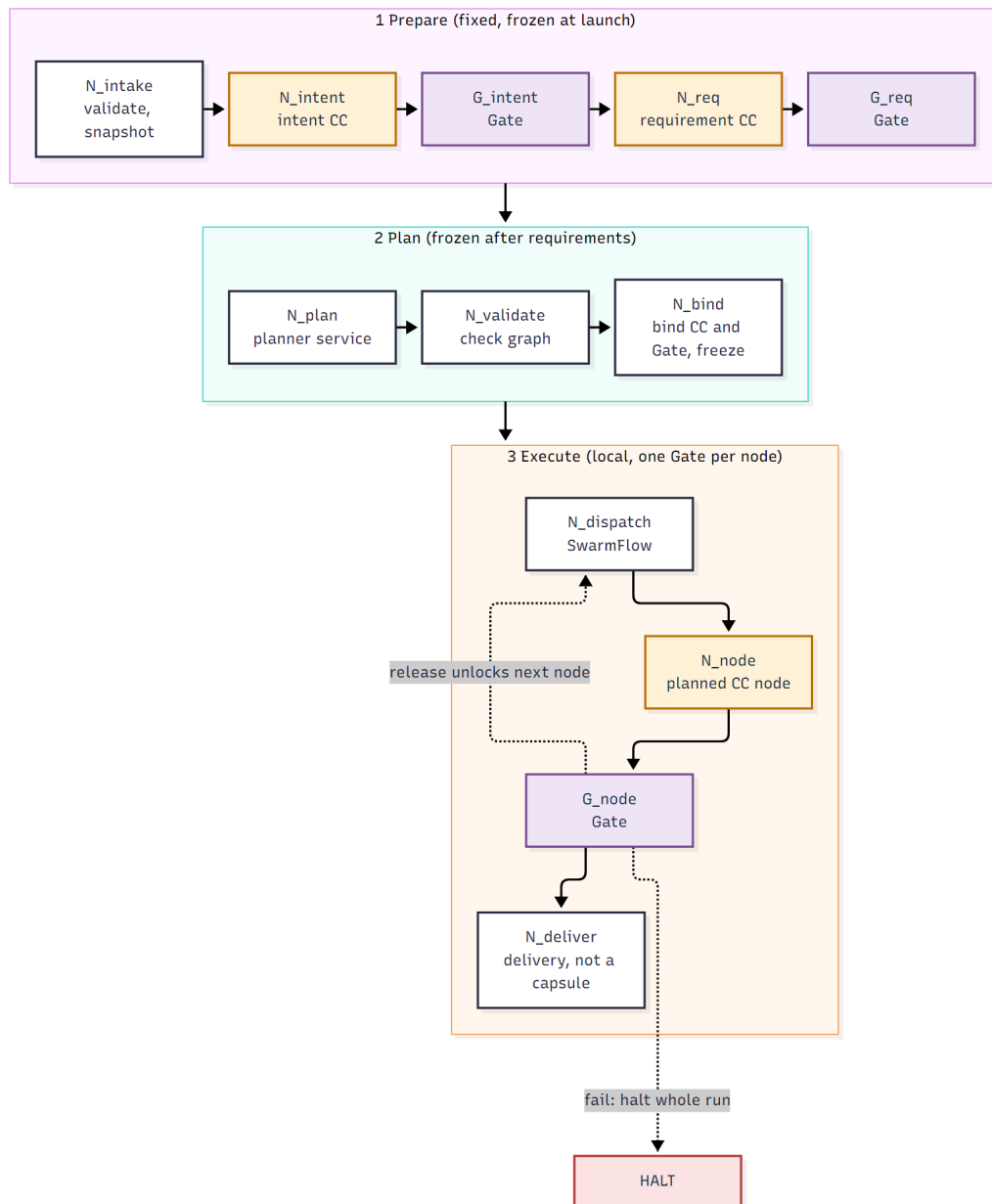
Piece	Capsule?	Why
Intent compiler (research.compile_intent)	yes	governed work (rules only for intent, model-backed for requirements), reusable, independently admitted
Requirement compiler (research.compile_brief)	yes	same
Research work capabilities (search, screening, hypothesis, POC, benchmark, evaluation, report writing)	yes	same. See capabilities
Search operators (op.*)	yes	reused by several callers; mediate external or repository access
Verifier (research.verifier)	yes	a CC. It assesses a node's result. Gate host code decides; the verifier only assesses.
Planner	no	service that proposes a DAG. Not admitted, not RSI-able.
Plan validator, binder, freeze	no	deterministic control code
Supervisor, runner, Gate host, store	no	fixed hosts. A capsule cannot run or judge itself.
Intake	no	validates the request and snapshots resources
Delivery	no	ordinary result processing and publication. No Declaration, no RSI.
Model bridge / router	no	ordinary service wrapping Codex. Never picks capsules.
Admission, library, librarian	no	not steps of a run

Test for a host: it knows nothing about any stage. If one stage's change needs a host change, the design is wrong.

Flow: data in, governed DAG, answer out

5 October 2026 | Specified design | Implementation validation pending

Overview. Fixed outer flow; planned nodes are fixed after freeze; every capsule call is followed by its Gate.



Failures

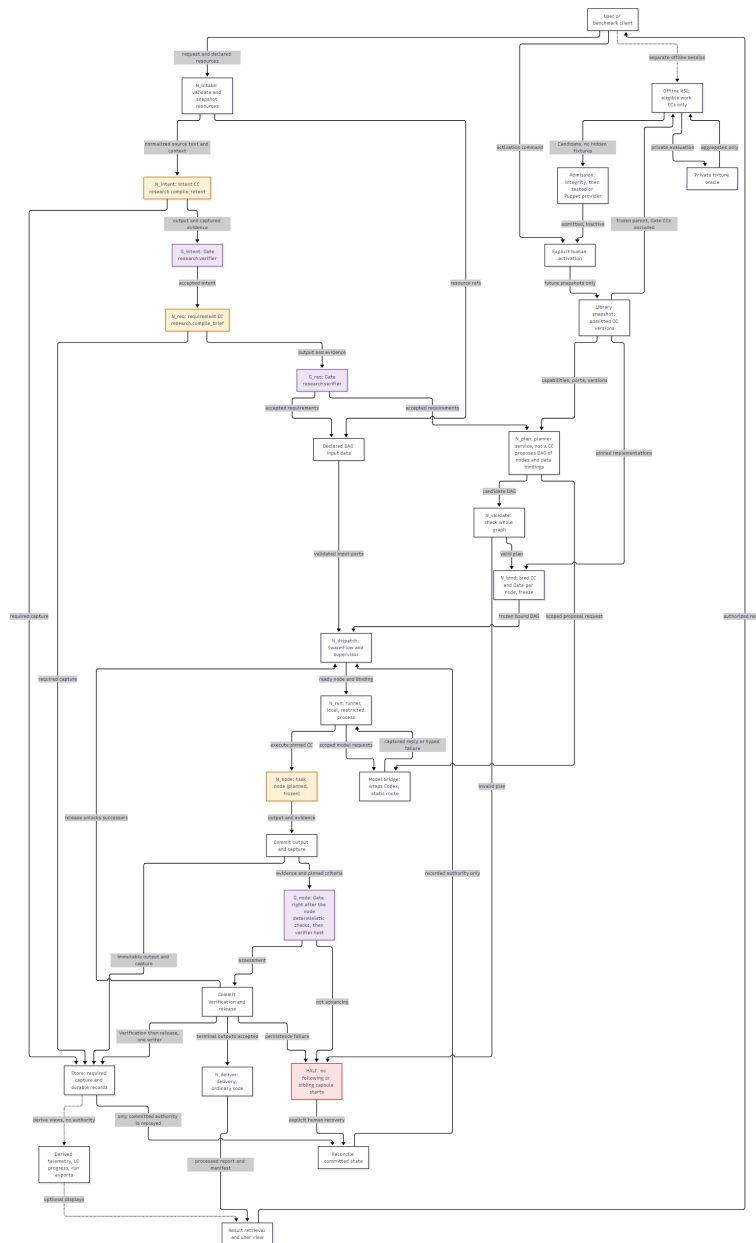
Failure	What happens	Owner
Intake invalid	rejected before any capsule runs	intake
Gate FAIL, ENVIRONMENT_BLOCKED, INCONCLUSIVE	halt whole run, keep committed evidence, route to human review	verification
Plan invalid (cycle, missing producer, wrong version, denied effect, budget)	zero dispatch, findings recorded	planner
Output commit or Verification commit fails	counts as failure; nothing is released	lifecycle
Timeout, cancel, denied effect	halt, kill child tree, keep capture	runtime
Duplicate request id	returns committed result or "in progress". Changed bytes = conflict.	lifecycle
Screening finds no eligible card	NO_ELIGIBLE_OPPORTUNITY , halt for human review (not an empty success)	screening
Valid negative science	not a failure. Report is still produced.	evaluation

Failure	What happens	Owner
Recovery	explicit, human-started. Reuse committed work, re-run missing Gate only. Changed inputs or pins need a new run.	lifecycle

Flow with observability and RSI shown

5 October 2026 | Specified design | Implementation validation pending

Full view. Observability lines are derived views only; RSI is a separate offline area.



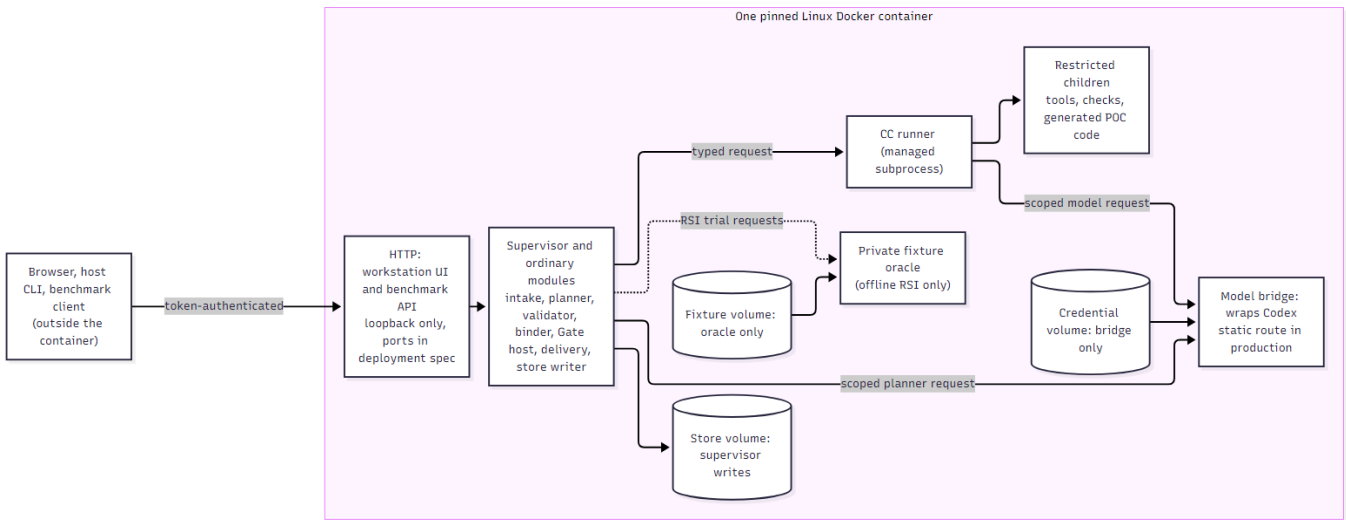
Placement and model routing

5 October 2026 | Specified design | Implementation validation pending

Decisions

- **One Docker container, one image: a modular monolith.** Control plane and restricted execution processes live in the same Linux image. Modules call each other as ordinary in-process calls. Separate OS processes exist only where trust must be separated.
- **Docker is packaging and the outer boundary.** It does not replace the inner boundaries for generated code or hidden fixtures. Capsule execution never starts another container. No Docker socket inside.
- **Model routing sits inside a model call, never above the plan.** The plan (or planner) picks CCs. A **model bridge** wraps Codex behind a provider interface and records which endpoint served each call. In M1 production the route is static: Codex. An isolated experiment may pick among approved endpoints; it cannot change which capsule runs or what a Gate decides.
- **Credentials stay with the bridge.** A dedicated persistent volume holds the Codex login, owned only by the bridge process. Capsules, prompts, exports and child processes never see it.

Placement graph



Model routing in one view

Question	Answer
Who chooses the capsule?	The plan. Never the router.
Who chooses the model?	Production: fixed Codex route. Experiment: router inside one call, from an approved list.
What does the bridge record?	requested and served model, route_id , joined to the call by obs_id . Unknown tokens/cost are null, not zero.
What does the model receive?	The prompt only. Stage, role and capsule labels stay on the trusted side.
What if the router is off or fails?	Use the pinned default if compatible; otherwise typed failure. No silent failover after an effectful call began.
Can routing change a Gate?	No.

Router team design is kept verbatim in model_router_design_en.md. Our adaptation is model-routing.

Verification: Gates, verifier CC, test policy

5 October 2026 | Specified design | Implementation validation pending

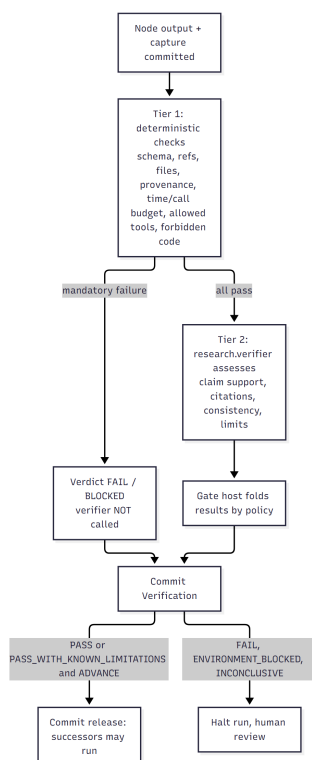
Rules

1. **The verifier is a CC.** `research.verifier` is one admitted capability. It is reused at every Gate. It takes one **evidence_bundle** and returns one **verifier_assessment**. It cannot release anything.
2. **A Gate follows every capsule call.** Intent call, the requirement call, each planned node. Nothing consumes an ungated output.
3. **The verifier checks the result of the node.** Not the library entry. The node is the task-specific use of a CC; the verifier sees its actual output and captured effects.
4. **Tests come from the Declaration.** A trusted builder compiles a test for each node from the producer's Declaration (output schema, guarantees, checks, effects, permissions, allowed failures) plus mandatory host policy. It places the Gate right after the node. A Declaration obligation the builder cannot test fails freeze. Nothing is silently skipped.
5. **Gates stop or change control flow.** Only a committed advancing Verification plus a committed release lets a successor start.
6. **Gate-role CCs have zero RSI-mutable components.** `evolution.rsi: none`, `evolution.may_change: []`. M1 RSI cannot touch the verifier's prompt, body, checks, dependencies or criteria. So "capsule X has success rate Y" compares against a fixed referee. A human referee revision starts a new comparison cohort.
7. **Nothing judges itself.** The Gate host validates the verifier's own answer directly; the verifier's call is not itself Gated.

Count and scope

Question	Answer
How many verifier CCs?	One identity, <code>research.verifier</code>. Many call sites reuse it through pinned criteria profiles.
How many Gates per run?	One per executed capsule call: 1 intent + one for the requirement call + one per planned node.
What does one Gate see?	Its producer's output, captured evidence, declared vs observed effects, pinned criteria. Not later steps.
Who decides?	Gate host code folds checks into one Verification. The verifier only assesses.
Where do criteria live?	Pinned Gate profiles (profiles); a producer cannot choose its own.

Two tiers



Verdicts

Verdict	Meaning	Run does
PASS	all mandatory checks passed	continues
PASS_WITH_KNOWN_LIMITATIONS	passed; limits recorded	continues, carries warnings
FAIL	mandatory defect	halt

Verdict	Meaning	Run does
ENVIRONMENT_BLOCKED	runtime or dependency problem, not an artifact defect	halt
INCONCLUSIVE	evidence insufficient	halt for review

Infrastructure verdict is not scientific verdict. A correctly produced scientific **FAIL** in an evaluation result gets infrastructure **PASS** and goes on to the report. Exact fold rules: Gate host. Fields: Verification.

Test policy

Quality comes from independent checks at every size of boundary. Nobody certifies their own work.

Level	What is tested	How	Evidence that counts
Block	one module or CC through its public API with fake neighbours	fixed fixtures, injected faults, no live model	observed outputs and refusals
Seam	a producer's real output entering its consumer	independently derived producer and consumer fixtures; a wrong output must be refused	both sides agree, wrong input refused
System	request to answer through the fixed flow and a small frozen DAG	small local baseline and recorded model replies, then real endpoint	run record shows every Gate and release

Rules:

- **Expected results come from someone other than the builder.** Fixture authors write check and rubric expectations.
- **Test the refusal, not only the pass.** Assert how many times the next node actually started, not just a verdict string.
- **Inject faults at fixed points:** after output commit, before/after Verification commit, before release, during cancel, killed child.
- **Unrun is unrun.** A check not executed, a missing service, a skipped probe or stale evidence is never reported as a pass.
- **Measure model work separately from deterministic work.** Compare correctness before latency or cost.
- **Documentation checks (lint, schema generation, link checks) show the design is consistent. They do not show the software works.**

Required cases (the ten Gate acceptance cases): valid pass; schema failure with Tier 2 not run; swapped or stale artifact; unsupported claim; budget violation; prohibited code or tool; environment block; known limitation; valid scientific negative result; delayed Gate keeps next node locked. Plus: decision computed but save failed; saved but reply lost; engine cache says pass but no committed Verification; kill after effect before record; repeated request id.

Where to find invocation points per component and the expected observations: test surfaces. Per-capability test lists live on each capability page. Build order ties each step to a check: build order.

Runtime, exceptions, permissions, observability

5 October 2026 | Specified design | Implementation validation pending

Who does what during a run

Part	Job	Never does
Supervisor (ordinary module)	validates, freezes, reserves each dispatch, calls runner, calls Gate host, commits release, dispatches next	run capsule code; decide a Gate
CC runner (managed subprocess)	loads the exact admitted capsule by hash, checks ports, runs it by kind, enforces call budget, captures output and effects, writes an Observation	choose a successor; talk to the store directly
Gate host	deterministic checks, calls the verifier, folds one Verification	invent criteria; let a capsule override the fold
Store	atomic, append-only durable records. Supervisor is the only writer.	n/a
Model bridge	one model turn per request, authenticated, captured	pick capsules

Details: runner, lifecycle, Gate host, storage.

One call, in order

1. Supervisor reserves a dispatch identity: (**run_id**, **step_id**, **binding_sha256**, **input hashes**, **attempt**). Reserved before anything runs.
2. Runner resolves the Binding, checks input types and preconditions, reserves model-call budget.
3. Runner executes by kind (**tool**, **skill**, **prompt_section** are the M1 kinds). Model turns go to the bridge; effectful work stays in a restricted child.
4. Output and raw capture are **committed** with an Observation.
5. Gate host evaluates; Verification is **committed**.
6. Supervisor re-reads committed bytes, checks **ADVANCE**, commits the **release**, dispatches the next node.

A passing decision whose save fails never releases anything.

Exception handling

Situation	Outcome	Recovery
Capsule output invalid or contract broken	Gate FAIL, halt	new run with changed implementation or input
Timeout (runner, bridge, process service, or outer deadline: first wins)	halt, kill the whole child tree, record which clock expired	explicit reviewed new attempt; never automatic
Cancel	kill and reap children; no new work	explicit
Denied effect, forbidden import or tool	FAIL, halt	fix and new run
Runtime or dependency unavailable	ENVIRONMENT_BLOCKED, halt	human fixes environment, new attempt, same pins
Process died after a possible effect, no Observation	uncertain effect; halt	human cleans attempt workspace, records review, new attempt
Duplicate request, same bytes	returns in-flight or committed result	none needed
Duplicate request, changed bytes	REQUEST_CONFLICT	caller error
Persistence failure	halt, nothing released	explicit
Model auth lost	halt call, keep capture	login, then explicit restart. No silent resubmit of a paid turn.

Rules: **zero autonomous retries** in M1 for every effect class. A failed run is kept as a clean failure record. Halt opens the native human session; in headless mode it writes the same evidence, opens nothing, and exits with code 3 (launch rejection 2, success 0). Gate policy is identical either way. Full rules: lifecycle.

Failure reasons use one registry (policy), not a vocabulary per capsule.

Permissions and credentials

- **A capsule gets only what its Declaration says.** Effects, network, dependencies and secrets are declared, mapped to enforceable rules, and compared with what was observed. Fields nothing can enforce are marked **unsupported** and fail closed (permissions).
- **Restricted children:** distinct non-root identity, private network namespace with no external interface, no credentials, no store, no fixtures, one writable attempt directory (process boundary).
- **Generated POC code is untrusted.** Runs only in a separate restricted service with offline, hash-pinned dependencies (process boundary).
- **Codex login** is bridge-owned on its own volume; never in prompts, configs, exports or child mounts (model auth).
- **Local access:** loopback ports only, per-start session token with **0600** file permissions, no external binding (environment).
- **Configuration** is resolved and frozen at run start; later edits cannot change a running run (environment).

Observability

- **Required capture is part of the run, not a display:** prompts and replies as sent, outputs, raw stdout/stderr, observed effects, Verification, release. If required capture is lost, the run blocks.

- **Join key: `obs_id`** ties everything inside one capsule call. Events, traces, UI progress, scorecards and benchmark exports are **derived views** that can be rebuilt from committed records.
- Hiding a view in a diagram or UI never turns capture off. Data Foundation's run records are a view over these records (observability, run records).
- Token and cost counts are recorded only when the endpoint reports them; otherwise explicit "unavailable".
- Static host facts (OS, CPU count, GPU model) are captured once at run start. No live CPU/GPU sampling in M1.

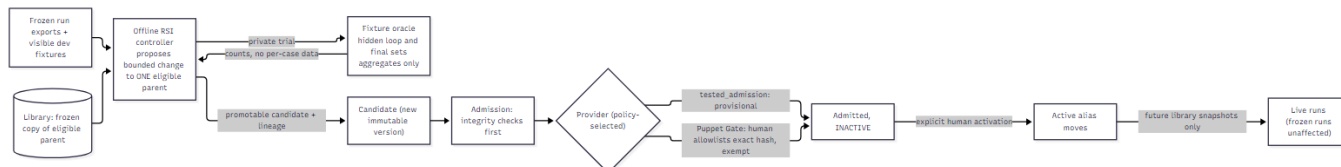
RSI: the separate improvement area

5 October 2026 | Specified design | Implementation validation pending

Position

- RSI is **offline**. It is a separate area with its own flow. It never runs inside a user run and never edits a live DAG, a frozen run, or an active capsule.
- It **brings candidate capsule versions in** through the same admission path as a hand-written capsule. A human controls what enters (the **Puppet Gate**) and what becomes active.
- The referee does not change. Verifier CCs (Gates), checks, hidden fixtures, scoring and activation tools are not RSI targets.

Flow



What may change

Rule	Value in M1
Eligible target	A capsule with evolution.rsi: propose and a non-empty may_change list. Gate-role CCs: none , empty list.
M1 targets	Target 1 (required) : the pure rank_opportunities helper in a sandbox copy of Screening. Target 2 (conditional) : Screening's own prompt and rubric text, only if a bounded headless model path exists.
Always frozen	Ports, interfaces, effects, permissions, dependencies, model selection, budgets, checks, verifier files, schemas, fixtures, split manifests, oracle code, scoring and acceptance policy, activation tools
Never	model weights, new capability types, self-permissions, live-run mutation, auto promotion
Output	a Candidate with parent hash. The parent is never edited.

Violations (out-of-scope mutation, hidden-fixture access, referee tampering, permission widening) halt the RSI session, keep evidence and need human clearance. Full rules: RSI engine, boundaries.

Benchmarks per capsule

Benchmarks are defined per capability before RSI begins. Mechanical, semantic and boundary fixture families are authored by someone other than the capsule author. Public datasets give visible development checks; the **hidden** sets belong to the oracle custodian.

Capsule	M1 RSI	Benchmark family to start from	Measured separately
research.select_opportunity (rank helper, text)	yes (Targets 1, 2)	ties, boundary scores, ineligible-but-high-score, empty eligible set; semantic score labels by an owner	correctness first; then elapsed time; model calls
research.compile_intent	none (PRD whitelist names Screening only)	intent span fidelity, contradictions, injection in request text	model calls vs deterministic work
research.compile_brief	none	target/comparator/quote fidelity, missing and contradictory limits	same
research.search_ideas, op.*	none	grounded-idea and citation cases; retrieval replay	retrieval vs synthesis
research.form_hypothesis, build_poc, run_benchmark, evaluate_results, write_report	none	see improvement guide	per stage
research.verifier and every Gate	never	planted-defect fixtures per Gate profile	n/a

Even where RSI is **none**, the benchmark families exist so a human can compare manual revisions on the same fixtures. Fixture sources and custody: benchmark material.

Admission and the Puppet Gate

- **Admission always runs integrity first**: Declaration hashes, types, dependency closure, permissions. Fail here and no provider is asked.
- **tested_admission** runs visible suites and records only checks actually executed. Grants **provisional**.
- **Puppet Gate (puppet_admission)**: a developer lists exact **decl_hash** values with actor and reason. Grants **exempt**. Runs no tests and claims none. It cannot write a Verification, release a node, change permissions, grant **certified** or activate an RSI child.
- **Version, assurance and active alias are separate**. An RSI child is always **admitted_inactive**. Activation is a separate human command that writes a record and moves the alias for future snapshots. Rollback is the same command pointing at an older admitted hash.

Detail: admission, library.

Build order and how docs map to code

5 October 2026 | Specified design | Implementation validation pending

Steps

#	Build	Needs	Exit check	Owner docs
0	Runtime unblocker: verify and finish the Codex adapter, secure local channel, minimal config, local workspace dirs	none	one bounded model call returns through the adapter; no TCP listener; session credential required	model auth, environment
1	Foundation: store (atomic, append-only), hashing, type vocabulary and generated schemas, Declaration, admission, library snapshot, minimal dispatch reservation and system-record writer. Author the toy fixtures: one pure capsule folder, a two-node toy plan, a toy Gate profile.	0	toy capsule admitted; changed bytes refused until re-admitted; killed write leaves no partial record	storage, fields, admission, records, types
2	Runner + minimal freeze: write Bindings for the toy plan, run the toy capsule by hash through the dispatch caller, capture, write Observation	1	output matches port types; wrong input refused before run; missing Binding refused; timeout kills child tree	runner, toolchain
3	Verifier CC + Gate host: Node A -> Gate -> Node B, with a recorded model reply for the verifier	2	valid output releases B; invalid output keeps B at zero starts; failed save releases nothing; headless halt exits 3	Gate host, verification
4	Supervisor + SwarmFlow: freeze 1 (prep plan) at launch, walk it, durable release	3	kill between nodes and resume: no node reruns, no result lost	lifecycle, nodes
5	Intake + intent CC + intent Gate	4	request in; accepted intent_ir or halt; changed capsule hash refused	intent compile, intake
6	Requirement CC + requirement Gate	5	accepted research_brief ; quote-grounding check fails an invented requirement	requirement
7	Planner + plan validator + planned-DAG freeze (freeze 2) + data injection (planner emits one fixed template DAG)	6	toy end to end: request in, two small nodes, Gates, delivery out. Invalid plan: zero dispatches.	planner, run plan
8	Research capabilities, chain by chain: search + operators; screening; hypothesis; POC	7	per-capability wire checks, no-eligible-card halt, freeze before POC	capabilities
9	Scientific execution: restricted POC service, benchmark, evaluation, report writing	8	baseline/treatment on a small fixture; scientific FAIL still yields a report	process boundary, measurement
10	Delivery and workstation: publication, CLI, web, TUI, headless run, sealed export	9	one full local run to a report; interruption/restart shows same records	workstation, benchmark export
11	Offline RSI: oracle, controller, Candidate path, Puppet Gate, activation	10 (needs a stable parent)	attack suite blocked and logged; counters persist; no auto activation	rsi
12	Isolated experiments (non-blocking): model-planned DAG, router, Code Mode, alternate verifier model	10	experiment flags denied in production; mocks until access approved	experiments

Items still undecided for steps 0-7 (second-plan handoff, toy delivery contract, fake model bridge, minimal config, policy numbers) are listed in decisions. Steps 0-7 give the "put data in one end, get an answer out the other" system on toy capsules. Steps 8-10 swap the toy capsules for the research capabilities. Step 11 is required for M1 acceptance but does not touch the live path.

Working rules

- **Seams are proven by real output entering the consumer and by a wrong output being refused.** Two sides compiling is not proof.
- **The runner grows only as the next boundary needs it.** Observability grows the same way: records first, keyed by **obs_id**; events and views later.
- **Parallel work is fine against pinned contracts.** Platform probes that have not run block only the affected execution profile. Never claim an unsupported platform passed.

- **Keep algorithms inside modules.** Output meaning, frozen policy, permission boundaries and evidence order must survive any optimization. Measure model calls and deterministic work separately.
- **Add a capsule only for independent reuse, governance or a permission boundary.** Otherwise it is an ordinary module. Capsule count is an outcome.

Decisions, PRD deviations, open items

5 October 2026 | Specified design | Implementation validation pending

Current decisions

ID	Decision	Owner page
A1	Fixed outer flow in SwarmFlow: intake, intent CC + Gate, requirement CC + Gates, planner, validate/bind/freeze, dispatch, delivery. Planned nodes are fixed after freeze.	flow
A2	The planner is an ordinary service, not a CC. In M1 it emits one fixed template DAG through the full validator. Model-proposed DAGs exist only in the isolated experiment track.	planner
A3	One intent-compilation CC, research.compile_intent , built from the existing intent compiler. One pass, no dialogue.	intent compile
A4	Capsule count is an outcome of boundaries, not a target. Inventory is derived from the capability pages.	capabilities
A5	Delivery is ordinary code. Not a capsule.	terms
A6	The verifier is a CC (research.verifier), one identity, reused at every Gate through pinned profiles.	verification
A7	A Gate follows every capsule call. Tests are derived from the producer's Declaration.	verification
A8	A failed Gate halts the whole run, including sibling branches. Zero autonomous retries.	runtime
A9	Verification, then release, are committed before any successor dispatch. Supervisor is the only writer.	lifecycle
A10	Gate-role CCs have zero RSI-mutable components, so success rates compare against a fixed referee.	verification
A11	Scientific verdict and infrastructure verdict are separate. A valid negative result reaches the report.	evaluation
A12	One Docker container, modular monolith. Inner restricted processes for generated code, model bridge and fixture oracle.	placement
A13	Model routing sits inside a model call and never selects capsules. Production route is static Codex through a bridge.	model routing
A14	Codex login lives on its own persistent volume owned by the bridge, with its own login separate from any desktop Codex profile. Same single container.	model auth
A15	Field tables are the single source of payload types and generate JSON Schema. Core types are closed; changes make a new version.	types
A16	obs_id joins everything inside a capsule call. Telemetry and Data Foundation records are derived views.	observability
A17	RSI is offline, targets only eligible work capsules, uses a private oracle and needs human activation.	rsi
A18	Puppet admission lets a developer allowlist exact hashes as exempt . It invents no test results and cannot release a node.	admission
A19	Version, assurance and active alias are separate. Activation changes future snapshots only.	library
A20	Build in thin slices: one capsule, then one connected capsule, proving each seam.	build order
A21	SkillFuzz-style analysis of capsule-set interactions is deferred. Current checks prove interface compatibility, not semantic safety of every combination.	library
A22	One library snapshot is pinned per run at launch. The two preparation steps (intent, requirements) run from a fixed prep plan frozen at launch. The planned DAG is validated, bound and frozen after requirements, in the same run, from the same snapshot. Unconfirmed reconciliation : older pages froze one static plan at launch.	lifecycle, planner

ID	Decision	Owner page
A23	No automatic librarian in M1. Standing changes (suspend, deprecate, roll back, activate) are manual. certified assurance is not granted in M1.	library
A24	Delivery is ordinary code with a deterministic manifest check. It has no Gate. Nested operator calls inside a node use mechanical checks and are covered by the parent node's Gate.	delivery
A25	Gate API is gate(obs_ref) -> GateResult , called after the runner commits the Observation. Gate host and runner front-end (CcBackend) run in the supervisor process.	Gate host

PRD deviations

The PRD is **docs/product/prd-m1-full-2026-10-02.txt**. These were chosen to build the system as defined. Everything the PRD blacklists stays out unless listed. Map of clauses to design: prd-map.

#	PRD says	We do	Still respected
1	Phase 1 DAG is hardcoded; planner bypassed (2.7, 4.8)	A planner proposes the DAG once from accepted requirements; it is validated, bound and frozen before any dispatch. First version emits one fixed template.	no live restructuring, no replanning after failure, no parallel swarms
2	Dynamic intent compilation is Phase 2; Phase 1 is one fixed one-shot compiler (3.2, 4.7)	An intent CC derives intent first, then requirement CC. Both one-shot, existing compiler as base, no dialogue.	no interactive clarification, no dynamic workflow selection
3	Evaluator Gate between governed stages (4.2)	Gate after every capsule call, built from the Declaration; verifier is itself a CC with zero RSI mutability	two-tier gate, read-only verifier, frozen referee
4	Native process sandbox, containers deferred (4.1.4, 5.4.3)	One Docker container plus inner restricted processes	no per-task containers, no Docker socket, no cluster
5	Human activation of RSI versions (4.4)	Adds a developer-controlled Puppet admission for exact hashes (exempt)	RSI children stay inactive until a human activates
6	Requirement compilation is one bounded LLM pass (3.2.1, 4.7)	The intent step is the existing rules-only compiler (no model call). The requirement step keeps the one model pass.	one-shot, no dialogue
7	One Brief from one compiler (3.2)	The prep plan has one requirement call. More requirement calls need a new decision.	single Brief to the planner
8	Strict admission with self-tests, one provisional level (4.1.2)	Two admission providers: tested (provisional) and Puppet (exempt)	no certified , no automatic librarian

Each deviation needs owner confirmation before the coding register treats it as final. Status: **unconfirmed**.

Open

Items architecture cannot settle alone. Each blocks only the work it names.

Pending source (do not invent)

Item	Affects	Note
Intent CC ports and wiring to the requirement CC. intent_ir exists; optional hint becomes a required accepted input.	steps 5-6	schemas likely unchanged; confirm when the CC is built
Gate profiles for intent and requirement call sites	verifier	profile contracts not written
Declaration-derived Gate builder: API and test-record shape	step 3, freeze	authority and placement are decided; shapes are not
Planner entry contract: services schema still takes a Brief and describes an experimental-only planner	step 7	revise with accepted intent and requirements
Intent CC RSI status	RSI	set to none (PRD names Screening only); revisit only by decision
Final CC count	inventory	follows from intent CC approach
Second-plan handoff: how the run script receives the planned DAG and pins after requirements; freeze-2 record kind and planned_plan_ref shape	step 7	lifecycle, records, runner
Plan wiring from accepted prep outputs (accepted.<prep_step>.<port>) into planned nodes; prep plan example; template without prep steps	step 7	run plan, template
Planner proposal envelope fields	step 7	planner

Item	Affects	Note
Toy fixtures for steps 1-7: one pure capsule folder, a two-node toy plan, a toy Gate profile, a type-agnostic delivery contract and what "answer out" is for a toy	steps 1-7	build order
Fake model bridge and replay file for system tests; whether a toy node may use a deterministic-only Gate profile	steps 3-7	test surfaces
First-epoch policy example with the runner and check numbers; minimal config keys per build step	steps 0-3	environment
Crash resume versus human-review gating, and a test-only review injection; store fault-injection seam	steps 3-4	lifecycle, storage
Bridge request/result table and mapping from the runner model call	step 0	environment
Gate profile id convention and profile hash rule	steps 3-6	verification
Experimental compiler requirement output type; generalist settings (future)	isolated track	experiments

Validation not yet run

Item	Acceptance evidence
Nested user-namespace and seccomp profile for generated code on Linux Docker Engine and macOS Docker Desktop	doctor probes pass; otherwise UNSUPPORTED_SECURITY_PROFILE and blocked
RSI adversarial suite (23 cases, repeated)	all blocked, logged, reconcilable
Native workflow spans for CC calls without a team worker	one run showing the join to CC records
Native web run view renders a non-team run	first integrated run
Existing intent compiler conforms to the intent type	component invocation
Hash-pinned offline wheelhouse per supported platform	clean offline install
Effect enforcement per backend (denied, mediated, observed, unsupported)	enforcement matrix with injected violations
Each measurement method has an adapter, coverage and fixture	registry evidence; missing blocks only that experiment